

Network Configuration Management with NETCONF and YANG

Programmable Networks A.Y. 24/25

In a nutshell ...

NETCONF

- The Network Configuration Protocol (NETCONF) provides mechanisms to install, manipulate, and delete the configuration of network devices.
[RFC 6241]

YANG

- YANG is a data modeling language used to model configuration and state data manipulated by the Network Configuration Protocol (NETCONF), NETCONF remote procedure calls, and NETCONF notifications.
[RFC 6020]

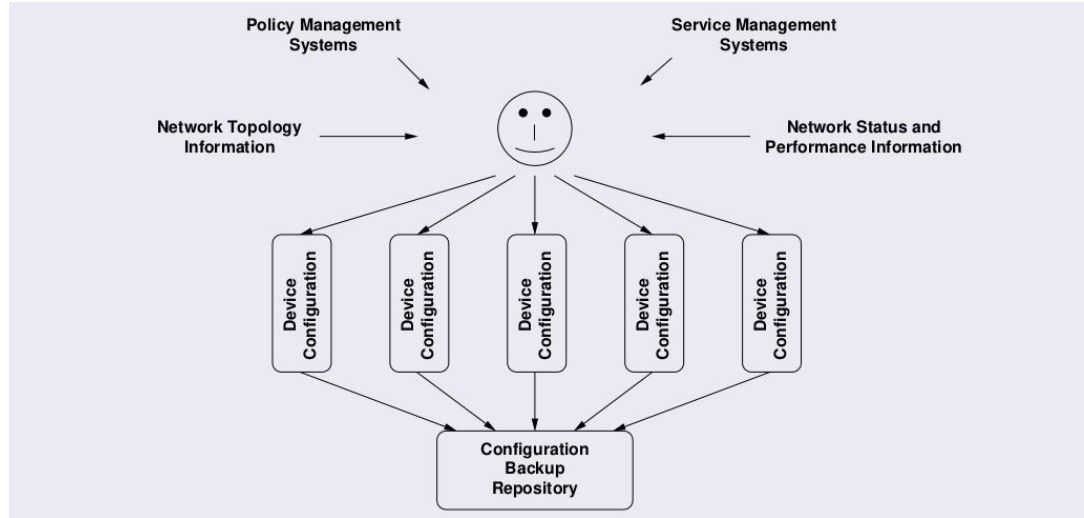
Outline

- Configuration Management
- Simple Network Management Protocol
- NETCONF Protocol
- YANG Data Modeling Language

Outline

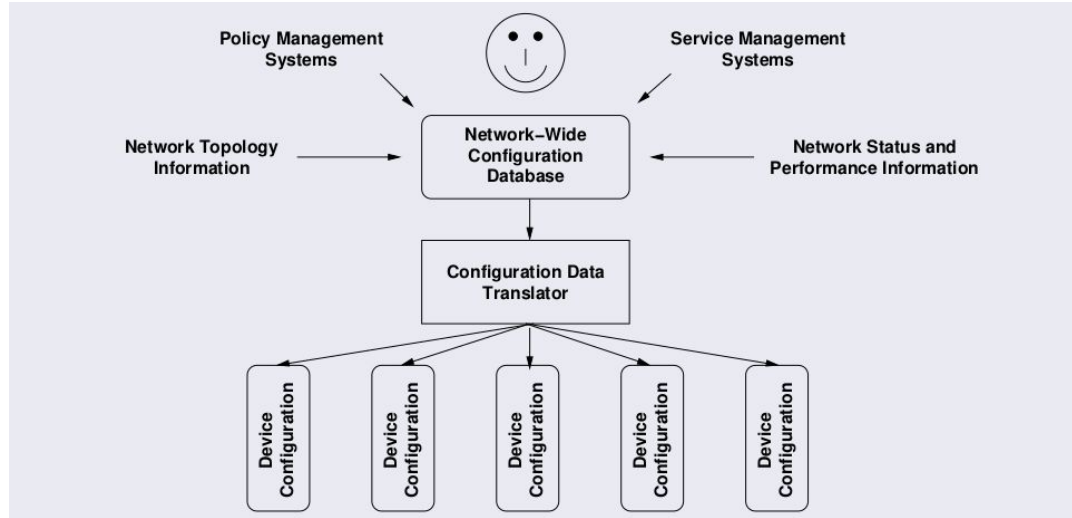
- Configuration Management
- Simple Network Management Protocol
- NETCONF Protocol
- YANG Data Modeling Language

“The Network is the Record”



- Network operators typing / scripting proprietary CLIs
- Often coupled with a backup repository to track changes
- Labor intensive, expensive, error prone, widely deployed

“Generate Everything” (RFC 3139)



- Changes go to a network-wide configuration database
- Device configurations are automatically pushed to devices
- Devices are (ideally) never touched manually

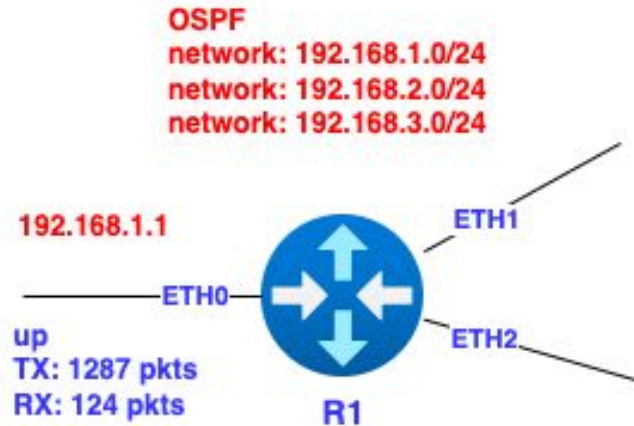
Configuration Management Requirements

R1: A configuration management protocol must be able to distinguish between configuration state and operational state. (configuration state vs. operational state)

- Configuration state:
 - Everything explicitly configured (e.g., IP addresses assigned manually to network interfaces)
- Operational state:
 - Usage and error counters obtained by the device
 - State learned from interaction with other devices (e.g., an IP address obtained from a DHCP server)

Configuration Management Requirements

R1: A configuration management protocol must be able to distinguish between configuration state and operational state. (configuration state vs. operational state)



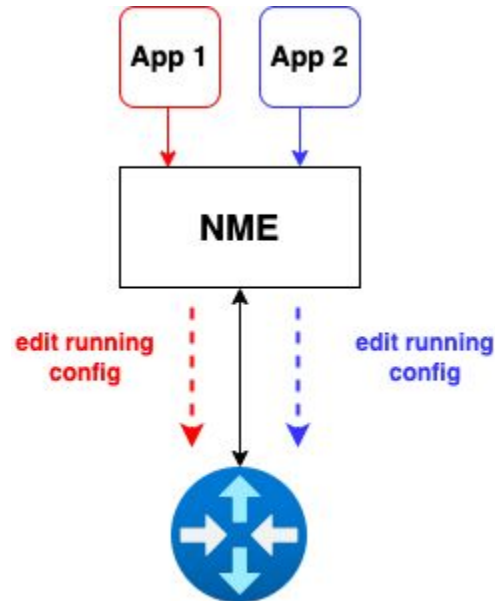
Configuration Management Requirements

R2: A configuration management protocol must provide primitives to prevent errors due to concurrent configuration changes. (concurrency support)

- Goal: Prevent two operators from making configuration changes simultaneously that lead to undesired results
- Requires either strict locking or a conflict resolution mechanism
- Computer programs generally deal better with strict locking, conflict resolution requires intelligence
- Scope of locks determines how much concurrency is possible

Configuration Management Requirements

R2: A configuration management protocol must provide primitives to prevent errors due to concurrent configuration changes. (concurrency support)



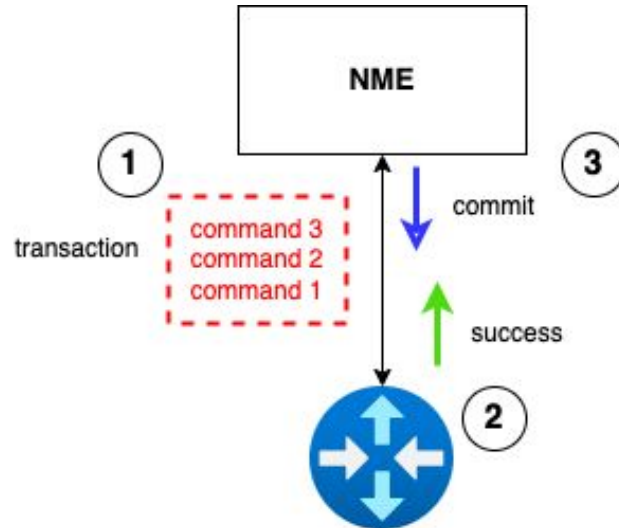
Configuration Management Requirements

R3: A configuration management protocol must provide primitives to apply configuration changes to a set of network elements in a robust and transaction-oriented way. (configuration transactions)

- Example: renumbering a whole network
 - All devices need to pick up new addresses
 - During the renumbering process (activation of a new configuration), connectivity might be temporarily lost
 - Devices need to be able to decide whether they keep the new configuration (addressing scheme) or rollback to the previous configuration

Configuration Management Requirements

R3: A configuration management protocol must provide primitives to apply configuration changes to a set of network elements in a robust and transaction-oriented way. (configuration transactions)



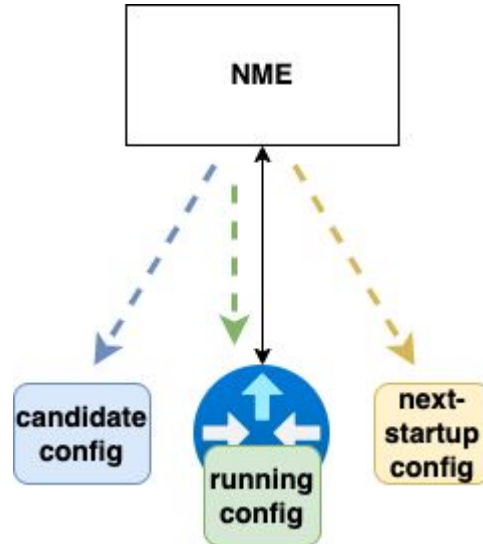
Configuration Management Requirements

R4: A configuration management protocol must be able to distinguish between several configurations and devices should be able to hold multiple configurations.
(multiple configurations)

- Configurations can become large and complex
- Often useful to keep a few configurations on the device to easily switch between them (backup configs)
- Distinction between the running configuration and the configuration used at next re-initialization

Configuration Management Requirements

R4: A configuration management protocol must be able to distinguish between several configurations and devices should be able to hold multiple configurations.
(multiple configurations)



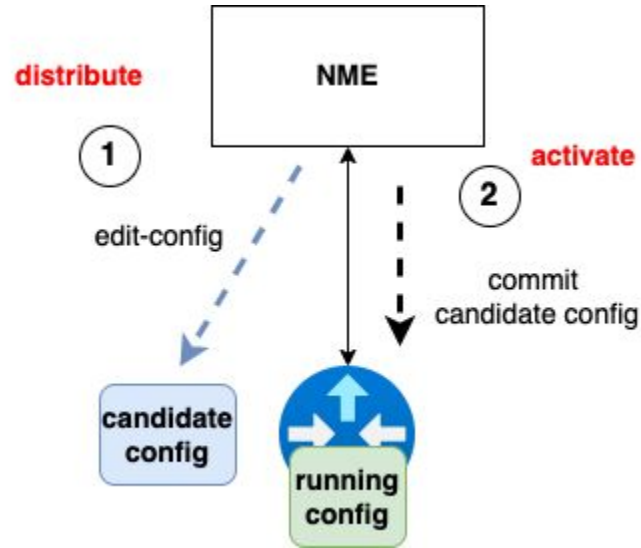
Configuration Management Requirements

R5: It is important to distinguish between the distribution of configurations and the activation of a certain configuration. (distribution vs. activation)

- Example: peak hours vs. off-peak hours
 - An operator wants to turn off links during off-peak hours
 - This might happen once per day or even more frequently
 - Instead of shipping a complete new configuration each time, it is much more efficient to activate a different already locally stored configuration

Configuration Management Requirements

R5: It is important to distinguish between the distribution of configurations and the activation of a certain configuration. (distribution vs. activation)



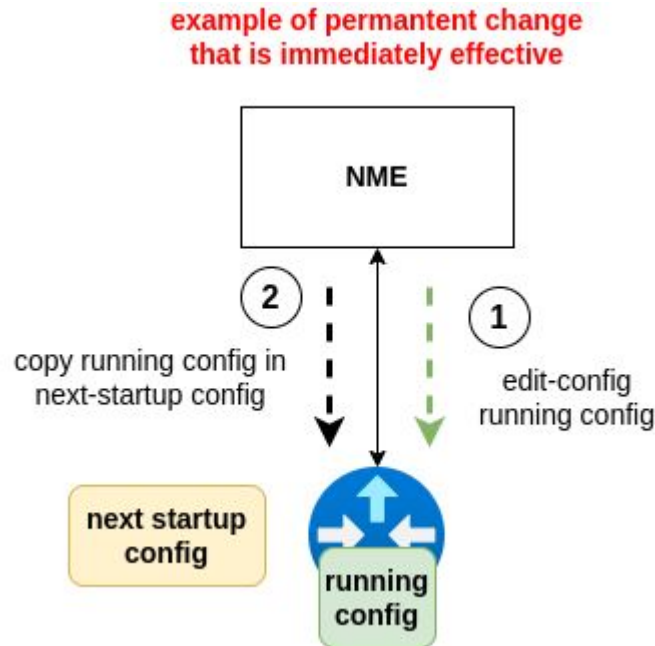
Configuration Management Requirements

R6: A configuration management protocol must be clear about the persistence of configuration changes. (persistence of configuration state)

- Sometimes changes are temporary and should be forgotten after the next restart of a device
- Sometimes changes should be applied immediately and be remembered after reboots
- Sometimes changes should only be applied at the next restart of a device

Configuration Management Requirements

R6: A configuration management protocol must be clear about the persistence of configuration changes. (persistence of configuration state)



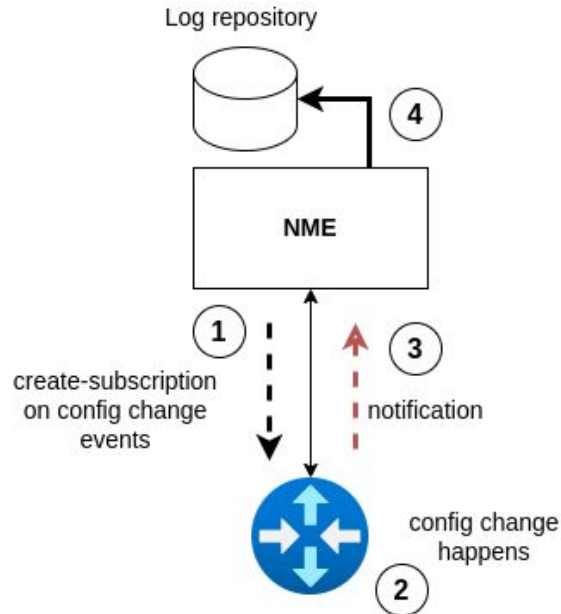
Configuration Management Requirements

R7: A configuration management protocol must be able to report configuration change events. (configuration change events)

- Example: firewall rules break applications
 - Customers report that certain applications suddenly fail to function correctly
 - Reason: A new firewall rule causes certain messages these applications rely on to be blocked
- A log of configuration change events often helps during fault isolation / resolution

Configuration Management Requirements

R7: A configuration management protocol must be able to report configuration change events. (configuration change events)



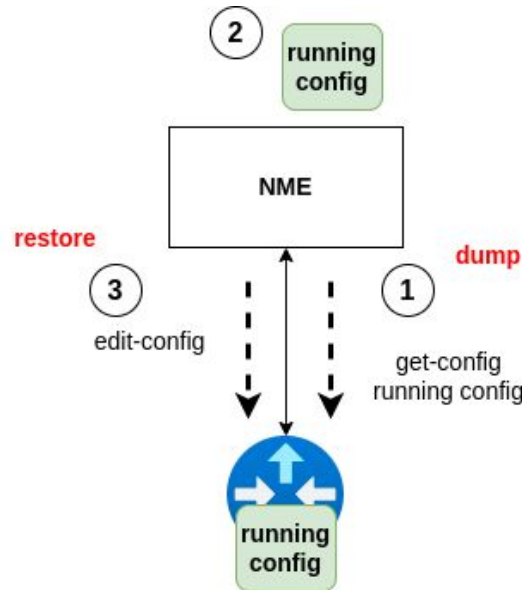
Configuration Management Requirements

R8: A full configuration dump and a full configuration restore are primitive operations frequently used by operators and must be supported appropriately. (configuration dump and restore)

- This seems to be rather obvious to operators
- Some network management protocols fail badly here

Configuration Management Requirements

R8: A full configuration dump and a full configuration restore are primitive operations frequently used by operators and must be supported appropriately.
(configuration dump and restore)



Configuration Management Requirements

R9: A configuration management protocol must represent configuration state and operational state in a form enabling the use of existing tools for comparison, conversion, and versioning. (support for standard tools)

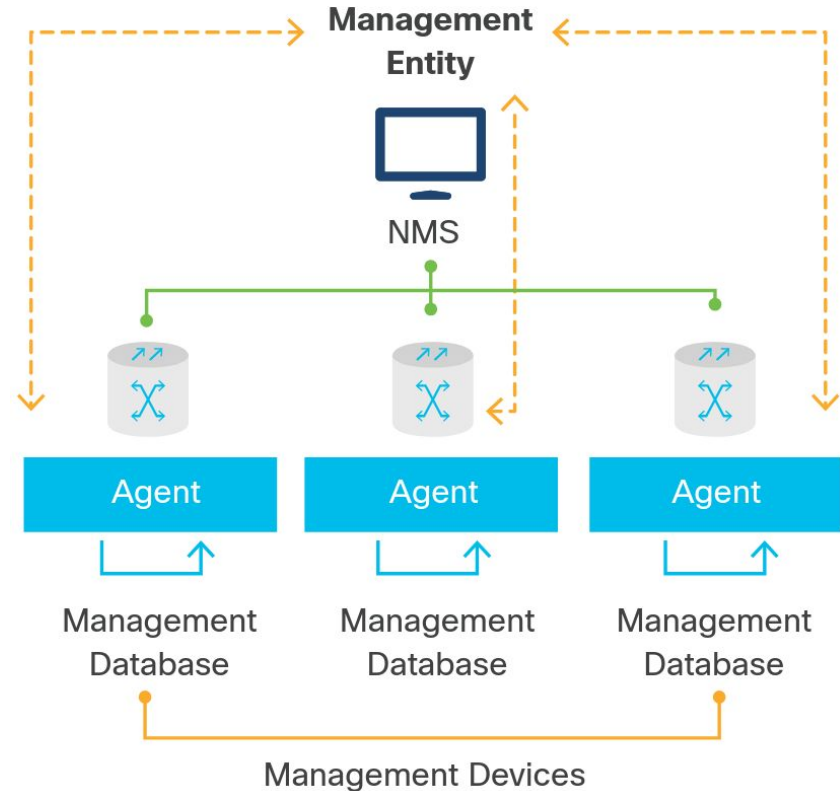
- Many operators use home-grown software systems
- Such systems often rely on standard tools for processing network management data
- Data formats enabling the use of off the shelf tools
 - eases integration
 - saves time
 - cuts costs

Outline

- Configuration Management
- Simple Network Management Protocol
- NETCONF Protocol
- YANG Data Modeling Language

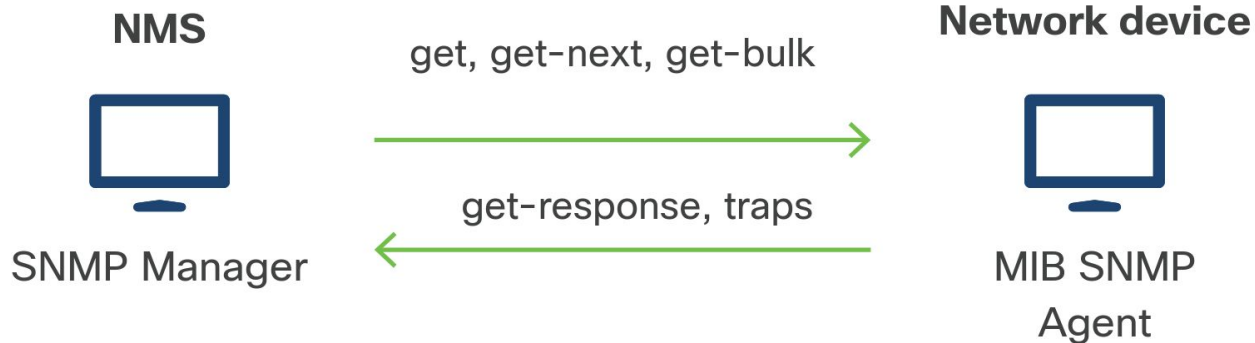
SNMP Components

- The SNMP system consists of **three elements**:
 - SNMP manager: network management system (NMS)
 - SNMP agents (managed device)
 - Management Information Base (MIB)



SNMP protocol

- **Managed devices** contain **managed objects** whose data is gathered into a **Management Information Base (MIB)**
- Two ways to convey MIB info, commands:
 - request/response mode
 - trap mode



SNMP protocol: message types

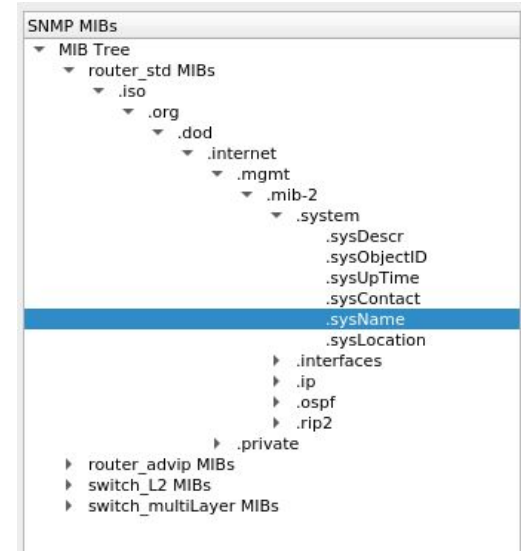
<u>Message type</u>	<u>Function</u>
GetRequest GetNextRequest GetBulkRequest	manager-to-agent: “get me data” (data instance, next data in list, block of data)
InformRequest	manager-to-manager: here’s MIB value
SetRequest	manager-to-agent: set MIB value
Response	Agent-to-manager: value, response to Request
Trap	Agent-to-manager: inform manager of exceptional event

Polling VS Trap

- The NMS can be configured to periodically have the **SNMP managers poll the SNMP agents** that are residing on managed devices using the get request
- Periodic SNMP polling does have **disadvantages**
 - **delay** between the time that an event occurs and the time that it is noticed
 - **trade-off** between polling **frequency** and **bandwidth usage**
- **Traps** are **unsolicited messages** alerting the SNMP manager to a condition or event on the network
 - some examples: improper user authentication, restarts, link status (up or down), MAC address tracking, closing of a TCP connection, loss of connection to a neighbor, etc.

Management Information Base (MIB)

- MIBs are **data structures that describe SNMP network elements** as a list of data objects
 - Think of the MIB as a "map" of all the components of a device that are being managed by SNMP
- The MIB is **organized in a tree-like structure** with unique **variables represented as terminal leaves**
- An **Object Identifier (OID)** is a long numeric tag, used to distinguish each variable uniquely in the MIB



Name/OID
.1.3.6.1.2.1.1.5.0
(.iso.org.dod.internet.mgmt.mib-2.system.sysName.0)

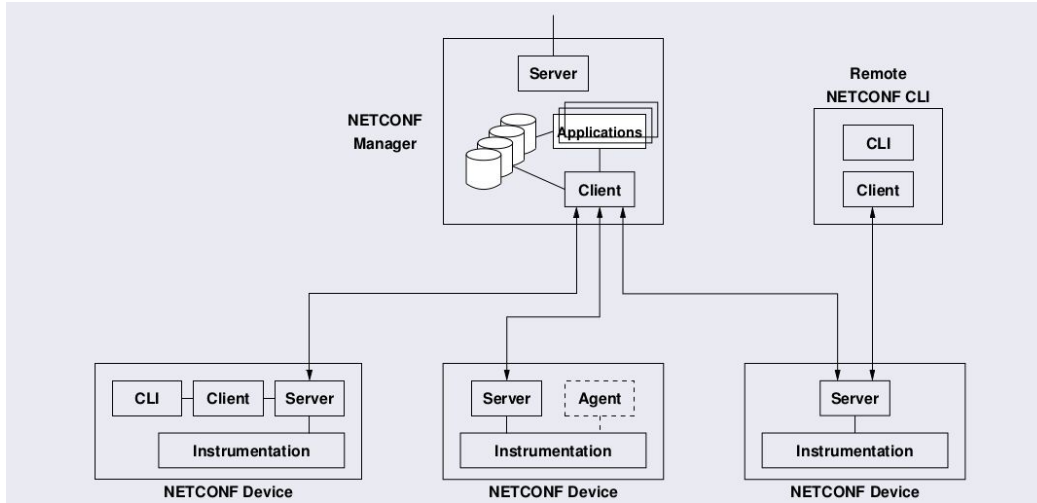
Outline

- Configuration Management
- Simple Network Management Protocol
- **NETCONF Protocol**
- YANG Data Modeling Language

SNMP Shortcomings

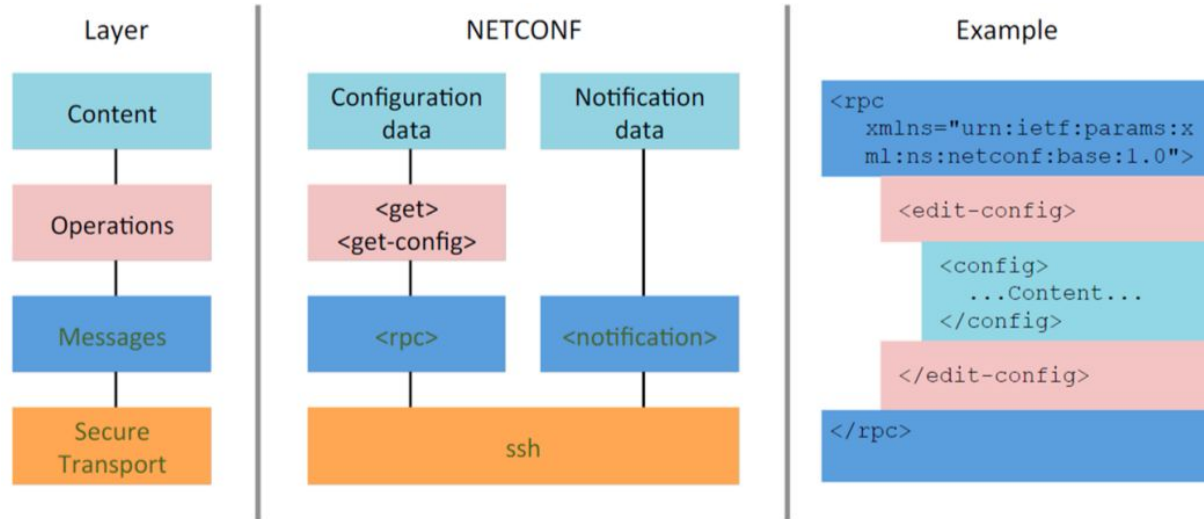
- SNMP:
 - Uses pull model when retrieving data from device which **does not scale up well** for high-density platforms
 - Does not have a discovery process for finding **Management Information Base (MIBs) supported by a device**
 - Does not support the concept of **transactions**
 - Lacks **backup-and-restore** of element configuration
 - **Limited industry support** for configuration MIBs

Deployment Model



- NETCONF enabled devices include a NETCONF server
- Management applications include a NETCONF client
- Device CLIs can be wrapped around a NETCONF client

Layering Model (RFC6241)



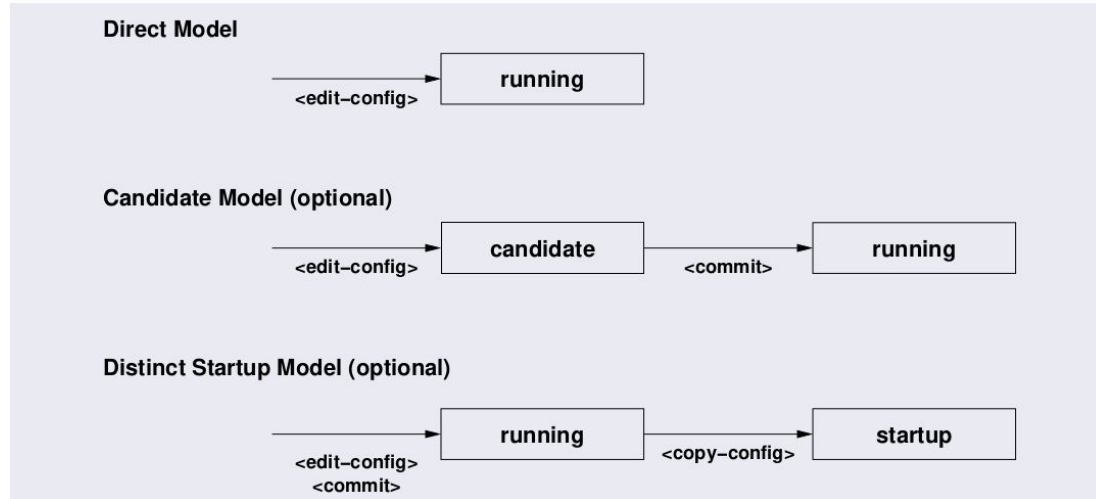
- Security is provided by the transport layer
- Operations focus on primitives to manage configurations
- The set of operations is designed to be extensible

Configuration Datastores (RFC6241)

A configuration datastore is the complete set of configuration information that is required to get a device from its initial default state into a desired operational state.

- The `<running>` configuration datastore represents the currently active configuration of a device and is always present
- The `<startup>` configuration datastore represents the configuration that will be used during the next startup
- The `<candidate>` configuration datastore represents a configuration that may become a `<running>` configuration through an explicit commit

Transaction Models (RFC6241)



Some operations (edit-config) may support different selectable error behaviours, including rollback behaviour

Capability Exchange (RFC6241)

After establishing a session over a secure transport, both NETCONF protocol engines send a hello message to announce their protocol capabilities, the supported data models, and the server's session identifier.

```
S: <hello xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
S:   <capabilities>
S:     <capability>
S:       urn:ietf:params:xml:ns:netconf:base:1.1
S:     </capability>
S:     <capability>
S:       urn:ietf:params:xml:ns:netconf:capability:startup:1.0
S:     </capability>
S:     <capability>
S:       urn:ietf:params:xml:ns:yang:ietf-interfaces?
S:       module=ietf-interfaces&revision=2012-04-29
S:     </capability>
S:   </capabilities>
S:   <session-id>4</session-id>
S: </hello>
```

Remote Procedure Calls (RFC6241)

The Remote Procedure Call (RPC) protocol consists of a `<rpc/>` message followed by an `<rpc-reply/>` message.

```
C: <rpc message-id="101"
C:      xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
C:   <get-config>
C:     <source>
C:       <running/>
C:     </source>
C:   </get-config>
C: </rpc>

S: <rpc-reply message-id="101"
S:      xmlns="urn:ietf:params:xml:ns:netconf:base:1.0"
S:   <data><!-- ...contents here... --></data>
S: </rpc-reply>
```

NETCONF Operations (RFC6241)

- `get-config(source, filter) → data`
 - Retrieve all or part of the configuration datastore source.
- `edit-config(target, default-operation, test-option, error-option, config)`
 - Edit the configuration datastore target by merging, replacing, creating, or deleting new config elements.
 - The test-option parameter allows to do a “dry run” while the error-option parameter controls how the server reacts to errors (stop, continue, rollback).
- `copy-config(target, source)`
 - Copy the content of the configuration datastore source to the configuration datastore target.

NETCONF Operations (RFC6241)

- `delete-config(target)`
 - Delete the named configuration datastore target.
- `lock(target)`
 - Lock the configuration datastore target.
- `unlock(target)`
 - Unlock the configuration datastore target.
- `validate(source)`
 - Validate the configuration datastore source (:validate capability).

NETCONF Operations (RFC6241)

- `get(filter) → data`
 - Retrieve all or part of the running configuration data store and merged with the device's state information.
- `close-session()`
 - Gracefully close the current session.
- `kill-session(session-id)`
 - Force the termination of the session session-id.

NETCONF Operations (RFC6241)

- `discard-changes()`
 - Revert the candidate configuration datastore to the running configuration (:candidate capability).
- `commit(confirmed, confirm-timeout, persist, persist-id)`
 - Commit the candidate to the running configuration datastore with an optional automatic rollback (:candidate, :confirmed-commit capability).
- `cancel-commit(persist-id)`
 - Cancel an ongoing confirmed commit identified by persist-id (:confirmed-commit capability).

Editing (Patching) Configuration

Embedded operation attributes specify how a configuration is modified by an `edit-config()` configuration.

- `merge`
 - Configuration data is merged with the configuration.
- `replace`
 - Configuration data replaces existing configuration.
- `create`
 - Configuration data is added iff it does not already exist.
- `delete`
 - Configuration data is deleted from the datastore.

Editing (Patching) Configuration Example

```
C: <rpc message-id="101"
C:       xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
C:   <edit-config>
C:     <target>
C:       <running/>
C:     </target>
C:     <config xmlns:nc="urn:ietf:params:xml:ns:netconf:base:1.0">
C:       <interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
C:         <interface nc:operation="replace">
C:           <name>Ethernet0/0</name>
C:           <mtu>1500</mtu>
C:         </interface>
C:       </interfaces>
C:     </config>
C:   </edit-config>
C: </rpc>
```

NETCONF over SSH (RFC6242)

Motivation: Use an already widely deployed security protocol for CLIs, reducing the costs associated with key management.

- SSH supports multiple logical channels over one transport layer association
- For framing purposes, the special end of message marker `] ]>] ]>` is used for the initial hello message
- Subsequent NETCONF 1.1 messages use a chunked framing format (if both parties use NETCONF 1.1)
- SSH is the mandatory to implement NETCONF transport

NETCONF over SSH: End-of-Message Framing

```
S:  <?xml version="1.0" encoding="UTF-8"?>
S:  <hello xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
S:    <capabilities>
S:      <capability>
S:        urn:ietf:params:xml:ns:netconf:base:1.1
S:      </capability>
S:      <capability>
S:        urn:ietf:params:xml:ns:netconf:capability:startup:1.0
S:      </capability>
S:      <capability>
S:        urn:ietf:params:xml:ns:yang:ietf-interfaces?
S:        module=ietf-interfaces&revision=2012-04-29
S:      </capability>
S:    </capabilities>
S:    <session-id>4<session-id>
S:  </hello>
S:  11>11>
```

Server announces:

- protocol version
- startup capability
- ietf-interfaces data model

NETCONF over SSH: End-of-Message Framing

```
C: <?xml version="1.0" encoding="UTF-8"?>
C: <hello xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
C:     <capabilities>
C:         <capability>
C:             urn:ietf:params:xml:ns:netconf:base:1.1
C:         </capability>
C:     </capabilities>
C: </hello>
C: ]]>]]>
```

- Client announces **protocol version**
- Client and server switch to chunked framing (both announced support for NETCONF 1.1)

Outline

- Configuration Management
- Simple Network Management Protocol
- NETCONF Protocol
- YANG Data Modeling Language

What is a Data Model?

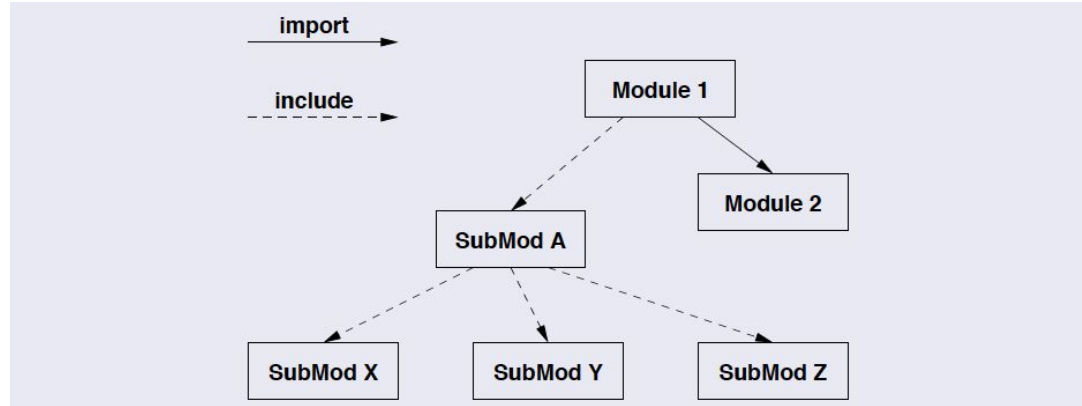
- A data model is simply a well understood and agreed upon method to describe "something"
 - As an example, consider this simple "data model" for a person
 - **Person**
 - Gender - male, female, other
 - Height - Feet/Inches or Meters
 - Weight - Pounds or Kilos
 - Hair Color - Brown, Blond, Black, Red, other
 - Eye Color - Brown, Blue, Green, Hazel, other

YANG

Data model language for the Network Configuration Protocol (NETCONF)

- Defined by the IETF in RFC 6020 in 2010
- YANG is an acronym for "Yet Another Next Generation"
- **Human readable**
- Any encoding format, including XML and JSON
- Transport over NETCONF over SSH, over gRPC, etc.
- YANG uses a compact C and Java-like syntax with high readability

Modules and submodules



- A module is a self-contained collection of YANG definitions
- A submodule is a partial module definition which contributes derived types, groupings, data nodes, RPCs, and notifications to a module

Module Example

```
module ietf-inet-types {
  namespace "urn:ietf:params:xml:ns:yang:ietf-inet-types";
  prefix "inet";

  organization
    "IETF NETMOD (NETCONF Data Modeling Language) Working Group";

  contact
    "WG Web: <http://tools.ietf.org/wg/netmod/>";

  description
    "This module contains a collection of generally useful derived YANG data types for
    Internet addresses and related things.";

  revision 2010-09-24 {
    description
      "Initial revision.";
    reference
      "RFC 6021: Common YANG Data Types";
  }
}
```

Built-in Data Types

Category	Types	Restrictions
Integral	<code>{u,}int{8,16,32,64}</code>	range
Decimals	<code>decimal64</code>	range, fraction-digits
String	<code>string</code>	length, pattern
Enumeration	<code>enumeration</code>	enum
Bool and Bits	<code>boolean, bits</code>	
Binary	<code>binary</code>	length
References	<code>leafref</code>	path
References	<code>identityref</code>	base
References	<code>instance-identifier</code>	
Other	<code>empty</code>	

Derived Types: typedef

```
module ietf-inet-types {
    namespace "urn:ietf:params:xml:ns:yang:ietf-inet-types";
    prefix "inet";

    typedef ipv4-address {
        type string {
            pattern '(([0-1]?[0-9]?[0-9]|2[0-4][0-9]|25[0-5])\.){3}'
            + '([0-1]?[0-9]?[0-9]|2[0-4][0-9]|25[0-5])'
            + '(%[\p{N}\p{L}]+)?';
        }
    }

    typedef ip-address {
        type union {
            type inet:ipv4-address;
            type inet:ipv6-address;
        }
    }
}
```

Data Nodes: Leafs, Leaf-lists, Container, Lists

- `leaf:`
 - A leaf has one value, no children, one instance.
- `leaf-list:`
 - A leaf-list has one value, no children, multiple instances.
- `container:`
 - A container has no value, holds related children, has one instance.
- `list:`
 - A list has no value, holds related children, has multiple instances, has a key property.

Example: leaf and leaf-list

```
leaf domain {  
    type inet:domain-name; // values are typed (type imported)  
    mandatory true; // must exist in a valid configuration  
    config true; // part of the set of configuration objects  
    description  
        "The host name of this system.";  
}
```

```
// XML: <domain>example.com</domain>
```

```
leaf-list search {  
    type inet:domain-name; // imported from the module with prefix inet  
    ordered-by user; // maintain the order given by the user  
    description  
        "List of domain names to search.";  
}
```

```
// XML: <search>eng.example.com</search>
```

```
// XML: <search>example.com</search>
```

Example: container

```
container system {
    config true;
    leaf hostname {
        type inet:domain-name;
    }
    container resolver {
        leaf domain { /* see above */ }
        leaf-list search { /* see above */ }
        description
            "The configuration of the resolver library.";
    }
}
```

```
// XML: <system>
// XML:   <hostname>server.example.com</hostname>
// XML:   <resolver>
// XML:     <domain>example.com</domain>
// XML:     <search>eng.example.com</search>
// XML:     <search>example.com</search>
// XML:   </resolver>
// XML: </system>
```


Example: list

```
list nameserver {  
    key address;  
    leaf address {  
        type inet:ip-address;  
    }  
    leaf status {  
        type enumeration {  
            enum enabled; enum disabled;  
        }  
    }  
}
```

```
// XML: <nameserver>  
// XML:   <address>192.0.2.1</address>  
// XML:   <status>enabled</status>  
// XML: </nameserver>  
// XML: <nameserver>  
// XML:   <address>192.0.2.2</address>  
// XML:   <status>disabled</status>  
// XML: </nameserver>
```